A netdiffuseR Tutorial: Estimating Network Influences on Behavior within the Diffusion Paradigm

George G. Vega Yon Aníbal L. Olivera M. Thomas W. Valente

George G. Vega Yon, University of Utah (corresponding author: george.vegayon@utah.edu)
Aníbal L. Olivera M., Universidad del Desarrollo
Thomas W. Valente, University of Southern California

Abstract

This paper introduces diffusion network research and presents an R library, netdiffuseR, that facilitates the estimation of contagion and diffusion processes on networks. Diffusion of innovations theory explains how new ideas and practices spread within and between communities. The spreading process consists of person-to-person interactions in which adopters persuade (by whatever means) non-adopters to adopt. Estimating these effects requires network data and an indicator for when each person adopts. The analytic steps involve creating an event history version of the data set and regression estimation to determine if exposure, the adopters in one's network, is associated with individual adoption. netdiffuseR automates many of these data management and processing steps, thus facilitating the estimation of these models. The package also contains simple ways to compute various network exposure measures such as based on structural equivalence, indirect ties, homophilic ties, etc. In addition, netdiffuseR provides capabilities to conduct simulations of network diffusion processes varying network structure, seed conditions, threshold distributions, and influence mechanisms. The paper includes example code for both empirical and simulated network studies. We aim to facilitate the adoption of netdiffuseR to advance the growth and evolution of diffusion network research.

Introduction

This paper provides a general overview of the R package netdiffuseR, a library for estimating contagion and diffusion processes on networks. We provide a brief introduction to diffusion

of innovations theory, introduce netdiffuseR, present a tutorial for empirical work, and give examples of simulations. We also describe ways to analyze the simultaneous diffusion of multiple behaviors, such as the adoption of complementary or competing products. Lastly, we demonstrate how to model disadoption. This paper enables researchers to use the netdiffuseR package to read, analyze, visualize, and simulate diffusion networks.

Diffusion of innovations is a theoretical approach that explains how new ideas and practices spread within and between communities. Diffusion of innovations theory has been used to study the spread of: fertility preferences and contraceptive choices (Bongaarts and Watkins 1996; Valente et al. 1997); risk behaviors such as smoking (Christakis and Fowler 2007); alcohol use (Light et al. 2013); substance use (Hoffman et al. 2006; Fujimoto and Valente 2013); physician behavior (Coleman et al. 1966; Iyengar et al. 2011); public health policies (Valente et al. 2015; Yamagata et al. 2017); physical activity (Aral and Nicolaides 2017); among other behaviors. Thus, developing a platform to easily analyze and compare factors associated with network influences on adoption has broad applicability in the behavioral and physical sciences.

Diffusion theory states that a new idea or practice enters a community or population from some external source such as an impersonal medium, a person from another community, or a technological shift (Rogers 2003; Valente and Rogers 1995). The idea or practice then spreads from person to person through interpersonal communication. Thus, the theory and techniques of social network analysis are integral to the study of the diffusion of innovations (Valente 2010), and constitute the field of diffusion networks.

The workhorse variable used to measure network influences on adoption is network exposure which is the number or proportion of adopters in a person's immediate social network contemporaneously or lagged by one (or more) time periods (which depending on the study could be days, months, or years). The formula for the lagged exposure model is:

$$E_{it} = \frac{\sum_{j \neq i} W_{ij} a_{j(t-1)}}{\sum_{j \neq i} W_{ij}} \tag{1}$$

Here, W_{ij} is a weight matrix indicating ties between individuals, most commonly a binary adjacency matrix such as a friendship network, although it can represent any relation. The adoption vector a indicates which of person i's alters j had adopted at time $t-1$, and the denominator is the row sum, or number of outgoing ties. E_{it} in Equation 1 measures the proportion of a person's ties at time t who had adopted the behavior by the prior time period. As described in Valente (2024), the exposure calculation can be extended by using a count (removing the denominator) or by weighting ties according to indirect connectivity, structural equivalence, strength (weak versus strong), homophily, Simmelian relations (alter-to-alter), or alter network properties such as centrality. The `exposure()` function enables users to calculate these alternatives through its arguments.

There are other measures needed to estimate diffusion network effects. A network threshold (Valente 1996) is the proportion or number of adopters in an individual's network required for

that individual to adopt. Although the observed exposure at adoption may coincide with this threshold, interval-based observation can make exposure overstate it. For example, a person whose threshold is two may be observed only after three peers adopted during the same interval; the observed exposure is then three even though the threshold was two (see Berry et al. 2019 for a detailed discussion).

Some people innovate with none or few adopters in their network adopting whereas others wait until a majority or all adopt. Network thresholds provide a means to estimate the classic two-step flow hypotheses (Katz 1957) and have been shown to moderate health promotion impacts (Valente and Saba 1998). In addition, the threshold distribution drives the diffusion rate and prevalence (Valente and Vega Yon 2020). Other diffusion network measures are Moran's I, a measure of contagion, network susceptibility, network influence (Valente et al. 2015), the hazard rate, and Bass model parameter coefficients (Bass 1969; Valente 1993).

The netdiffuseR package

The netdiffuseR R package (Valente and Vega Yon 2020; Vega Yon et al. 2026) was first released to CRAN on February 18, 2016. The package contains a collection of algorithms and methods first envisioned by Dr. Valente and released as GAUSS code with his book (Valente 1995).

In a nutshell, netdiffuseR provides methods for simulating, processing, analyzing, and visualizing diffusion networks. Among its key functions are the `survey_to_diffnet()` function, `rdiffnet()` function, and the `exposure()` function. The `survey_to_diffnet()` function provides an interface to read in tabular data (in R `data.frame` format) corresponding to survey data containing network nominations and time of adoption [TOA]. The function makes it easy to process survey data, pass multiple checks, and build the adoption and cumulative adoption matrices. The `rdiffnet()` function allows simulating diffusion networks using the threshold model and, since version 1.23.0, simulating multi-diffusion and disadoption. Users can simulate diffusion processes on pre-defined networks or random graphs in a computationally efficient way. Finally, the `exposure()` function–the workhorse of the R package–provides a flexible framework for calculating various types of exposure measures as indicated above. In the following section of the paper, we will provide a demonstration of these functions.

A tutorial on netdiffuseR

This tutorial was developed using **R** version 4.5.1 (R Core Team 2025), **Quarto** version 1.10.18 (Allaire et al. 2025), and **netdiffuseR** version 1.25.1. The multi-adoption and disadoption features demonstrated below were introduced in netdiffuseR version 1.23.0 and are available in the current CRAN release. Readers can find the Quarto document and all materials required

to reproduce the paper, including a container specification of the computing environment, at https://github.com/gvegayon/netdiffuser-connections.

Analyzing existing data

First, we will show how to read survey data into netdiffuseR. We start by loading the package. If you don't have the package installed in your system, you can use the `install.packages()` function:

```
install.packages("netdiffuseR")
```

Notice that this is required to do only once. The previous code will install the package in your R system, but it will not load it. Once you are sure the package is available in your computer, you can use the function `library()` to load it:

```
# Loading the netdiffuseR
suppressPackageStartupMessages(
  suppressWarnings(library(netdiffuseR))
)
```

The package includes three classical diffusion datasets that we can use as a starting point. Here, we use the Korean Family Planning study (Rogers and Kincaid 1981). Let us first inspect the data using the `data()`, `dim()`, and `str()` functions. The `data()` function loads datasets from R packages, `dim()` reports their dimensions, and `str()` summarizes their structure:

```
# Loading the dataset and taking a look
# at the data structure
data(kfamily)
dim(kfamily) # Number of rows and columns
```

```
[1] 1047  432
```

```
str(kfamily[,1:10]) # structure of the first 10 columns
```

```
'data.frame':   1047 obs. of  10 variables:
 $ village: num  1 1 1 1 1 1 1 1 1 1 ...
 $ id     : num  2 3 4 5 7 8 9 11 12 14 ...
 $ recno1 : int  1 1 1 1 1 1 1 1 1 1 ...
 $ studno1: int  1 1 1 1 1 1 1 1 1 1 ...
 $ area1  : int  1 1 1 1 1 1 1 1 1 1 ...
```

```
 $ id1    : int  2 3 4 5 7 8 9 11 12 14 ...
 $ nmage1 : int  0 0 0 0 0 0 0 0 0 0 ...
 $ nmage2 : int  1 0 2 0 0 0 0 0 0 0 ...
 $ nmage3 : int  0 0 0 0 0 1 0 1 1 0 ...
 $ nmage4 : int  0 0 0 0 0 1 0 0 1 1 ...
```

> **Tip**
>
> You can always inspect datasets that are included in an R package using the `data()` function. For example, in netdiffuseR, you can use `data(package="netdiffuseR")`.

As you can see, the dataset contains 1,047 observations with 432 columns. The dataset consists of survey data collected in 1973-1974 from women of childbearing age in 25 rural villages. For this example, we will focus on the two largest villages, namely villages 2 and 21. We can subset the data using the following code:

```r
kfamily <- subset(kfamily, village %in% c(2, 21))

# These identifiers and time variables represent whole numbers.
# Converting them explicitly prevents coercion warnings below.
kfamily[c("id", "village", "toa")] <- lapply(
  kfamily[c("id", "village", "toa")],
  as.integer
)
```

The survey has multiple network nomination questions. Depending on the situation, one may aggregate them or focus on one relation. We use the "talks to a neighbor" question, for which participants nominated up to five people. We then use `survey_to_diffnet()` to create a `diffnet` object:

```r
# Reading in the data
kfam_diffnet <- survey_to_diffnet(
  dat      = kfamily,
  idvar    = "id",
  netvars  = c("net11", "net12", "net13", "net14", "net15"),
  toavar   = "toa",
  groupvar = "village"
)
```

The key points are: (1) `idvar` identifies each individual in the dataset; (2) `netvars` lists the network-nomination columns; (3) `toavar` identifies the time-of-adoption variable, measured in years here; and (4) because the data contain multiple communities, `groupvar` specifies that ties

can occur within but not between villages. The function has additional options documented in `?survey_to_diffnet`.

R distinguishes between numeric values stored as decimals and values stored as integers. The identifiers, village codes, and adoption years in this example are whole numbers, so we converted them explicitly with `as.integer()` before calling `survey_to_diffnet()`. This prevents the coercion warnings that would otherwise appear without changing the values.

The terminal time category in the Korean Family Planning data requires special treatment. Period 11 does not represent a regular observation wave: instead, it assigns an adoption time to every respondent who had not adopted in an earlier period. If these assignments were treated as observed adoptions, cumulative prevalence would be forced to 100%, which could bias analyses of the diffusion process. In the two villages used here, 51 respondents receive this terminal code. We treat them as right-censored by replacing their time of adoption with `NA`, and then use the `diffnet` subsetting method to remove period 11. The resulting object contains the ten genuinely observed periods and a final observed prevalence below 100%:

```
# Identify respondents assigned to the terminal time category.
terminal_period <- max(kfam_diffnet$meta$pers)
terminal_cases <- diffnet.toa(kfam_diffnet) == terminal_period

# Treat their adoption times as right-censored.
diffnet.toa(kfam_diffnet)[terminal_cases] <- NA_integer_

# Retain only the genuinely observed network periods.
kfam_diffnet <- kfam_diffnet[, , kfam_diffnet$meta$pers < terminal_period]
```

The `diffnet` object we just created has several inspection methods. `print()` gives a glimpse of the data, including the number of time points and nodes, network type, and adoption prevalence. `summary()` describes the adoption dynamics, including the number and proportion of adopters at each time point, network density, and Moran's I (Moran 1950) as a measure of spatial autocorrelation. Finally, `plot_diffnet()` visualizes the diffusion process by coloring nodes as non-adopters, new adopters, or continuing adopters. The following code chunk demonstrates the first two functions:

```
print(kfam_diffnet) # Using print() is not necessary, but recommended.
```

```
Dynamic network of class -diffnet-
 Name               : Diffusion Network
 Behavior           : Unknown
 # of nodes         : 113 (201, 202, 203, 204, 205, 206, 207, 209, ...)
 # of time periods  : 10 (1 - 10)
 Type               : directed
```

```
 Num of behaviors   : 1
 Final prevalence   : 0.55
 Static attributes  : village, recno1, studno1, area1, id1, nmage1, nmag... (430)
 Dynamic attributes : -
```

```
summary(kfam_diffnet)
```

```
Diffusion network summary statistics
 Name     :  Diffusion Network
 Behavior : Unknown
```

Period	Adopters	Cum Adopt. (%)	Hazard Rate	Density	Moran's I (sd)	
1	6	6 (0.05)	-	0.02	0.01 (0.03)	
2	12	18 (0.16)	0.11	0.02	0.01 (0.03)	
3	7	25 (0.22)	0.07	0.02	0.04 (0.03)	
4	7	32 (0.28)	0.08	0.02	0.12 (0.03)	***
5	5	37 (0.33)	0.06	0.02	0.15 (0.03)	***
6	3	40 (0.35)	0.04	0.02	0.13 (0.03)	***
7	4	44 (0.39)	0.05	0.02	0.13 (0.03)	***
8	5	49 (0.43)	0.07	0.02	0.12 (0.03)	***
9	7	56 (0.50)	0.11	0.02	0.15 (0.03)	***
10	6	62 (0.55)	0.11	0.02	0.12 (0.03)	***

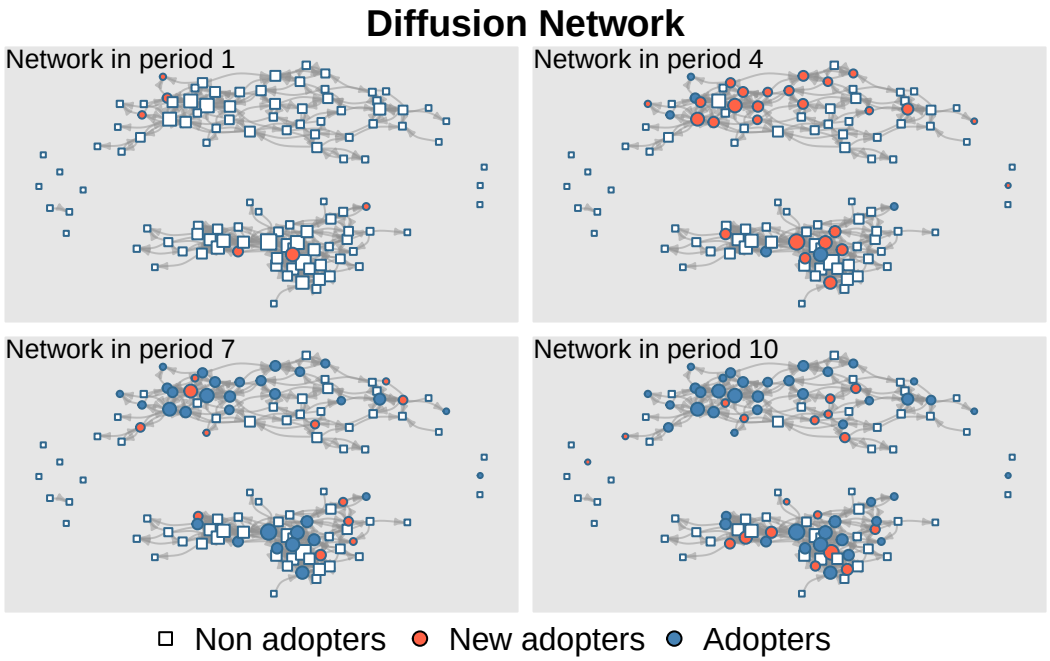
```
 Left censoring  : 0.05 (6)
 Right centoring : 0.45 (51)
 # of nodes      : 113

 Moran's I was computed on contemporaneous autocorrelation using 1/geodesic
 values. Significane levels  *** <= .01, ** <= .05, * <= .1.
```

> **Tip**
>
> By default, Moran's I is computed using the inverse of the geodesic-distance matrix. The `moran()` function also supports other weighting matrices.

Because the network layout is stochastic, we use `set.seed()` immediately before plotting to make the figure reproducible:

```
set.seed(33)
plot_diffnet(kfam_diffnet)
```



Of the observed outcomes, it is noteworthy that (a) the diffusion process is apparent in the figure, with adoptions occurring from the core toward the periphery of the graph, and (b) Moran's I shows a high level of spatial autocorrelation between periods four and ten. Other visualization functions include `plot_diffnet2()`, `plot_threshold()`, `plot_adopters()`, and `plot_infectsuscept()`. We invite readers to learn more about them in the netdiffuseR manual.

Computing exposures

Exposures are a fundamental part of estimating diffusion network effects. Consequently, netdiffuseR provides a wide range of possibilities when it comes to calculating exposures. Using the `exposure()` function, users can calculate cohesion (direct ties) and structural equivalence exposures (Burt 1987), attribute weighted exposure, tie-weighted exposure, count exposure (instead of proportion), and many more. The following code blocks show how to calculate three different types of exposure:

Cohesive exposure. This is the simplest type of exposure: the proportion of neighbors who have adopted at each time point. The `exposure()` function returns an $n \times T$ matrix, where n is the number of nodes and T is the number of time points:

```
exp_cohesive <- exposure(kfam_diffnet)

# Showing the exposure for the first five agents
```

```
# and first six time points
exp_cohesive[1:5, 1:6]
```

```
    1 2 3   4   5   6
201 0 0 0 1.0 1.0 1.0
202 0 0 0 0.5 0.5 0.5
203 0 0 0 0.0 0.0 0.0
204 0 0 0 1.0 1.0 1.0
205 0 0 0 0.0 0.0 0.0
```

Structural equivalence. This type of exposure is useful when influence may operate through similarity in network position rather than only through direct connections. An individual may be more likely to adopt a behavior when others occupying similar network positions adopt.

```
exp_se <- exposure(kfam_diffnet, alt.graph = "se", valued = TRUE)

# Showing the exposure for the first five agents
# and first six time points
exp_se[1:5, 1:6]
```

```
           1       2       3       4       5       6
201 0.006669 0.01595 0.02043 0.02411 0.03250 0.03614
202 0.006642 0.01583 0.02029 0.02369 0.03207 0.03571
203 0.006915 0.01742 0.02266 0.02829 0.03833 0.04234
204 0.006737 0.01580 0.02035 0.02363 0.03178 0.03547
205 0.007058 0.01807 0.02465 0.03171 0.04161 0.04553
```

Structural-equivalence scores are valued rather than binary, so we specify `valued = TRUE`.

Attribute weighted. Finally, this type of exposure can be used when a characteristic of adopters may make them more influential in the social contagion process. For example, people may be more influenced by older individuals than younger:

```
exp_age <- exposure(kfam_diffnet, attrs = "age")

# Showing the exposure for the first five agents
# and first six time points
exp_age[1:5, 1:6]
```

```
    1 2 3      4      5      6
201 0 0 0 1.0000 1.0000 1.0000
202 0 0 0 0.4795 0.4795 0.4795
203 0 0 0 0.0000 0.0000 0.0000
204 0 0 0 1.0000 1.0000 1.0000
205 0 0 0 0.0000 0.0000 0.0000
```

We can look at the three different exposures we just calculated by combining them into a single matrix. The following code chunk takes the fifth column (exposure at the fifth time point) and shows the exposures for the first ten individuals in the network:

```
# Visualizing exposure at time 5
cbind(
  "Cohesion"       = exp_cohesive[, 5],
  "SE"             = exp_se[, 5],
  "Attr. weighted" = exp_age[, 5]
) |> head(n = 10)
```

```
     Cohesion      SE Attr. weighted
201       1.0 0.03250         1.0000
202       0.5 0.03207         0.4795
203       0.0 0.03833         0.0000
204       1.0 0.03178         1.0000
205       0.0 0.04161         0.0000
206       0.5 0.03224         0.5000
207       0.0 0.27304         0.0000
209       0.0 0.27304         0.0000
210       1.0 0.03830         1.0000
2101      0.0 0.28641         0.0000
```

In addition to `exposure()`, netdiffuseR includes functions that can be combined with its core tools. For instance, we can use `vertex_covariate_compare()` to calculate age-homophilic exposure. Define an age-similarity weight as $s_{ij} = (1 + \|x_i - x_j\|)^{-1}$. Adding one keeps the weight finite and gives equal-age alters the maximum weight of one. The resulting exposure is:

$$E_{it} = \frac{\sum_{j \neq i} W_{ij} a_{jt} s_{ij}}{\sum_{j \neq i} W_{ij} s_{ij}}, \qquad s_{ij} = \frac{1}{1 + \|x_i - x_j\|}.$$

Here, $\|x_i - x_j\|$ is the L2 distance between the ages of individuals i and j. The following code uses `vertex_covariate_compare()` to generate an age-distance graph. Because the `graph` argument requires a single adjacency matrix, we pass the first graph in `kfam_diffnet`:

```r
# Using homophilic distance
age_distance_graph <- vertex_covariate_compare(
  # Passing the first graph
  graph   = kfam_diffnet$graph[[1]],
  # Passing the age parameter
  X       = kfam_diffnet[["age"]],
  funname = "distance"
)
```

The expression `kfam_diffnet$graph[[1]]` accesses the first network in the `diffnet` object, while `kfam_diffnet[["age"]]` extracts the `age` attribute. We can inspect the first few entries of the distance matrix:

```r
age_distance_graph[1:10, 1:10]
```

```
10 x 10 sparse Matrix of class "dgCMatrix"

 [1,] . 4 . 3 . . . . . .
 [2,] 4 . . 1 . . . . . .
 [3,] . . . . . . . . . .
 [4,] . 1 . . . 1 . . . .
 [5,] . . . . . . . . . .
 [6,] 4 . . 1 . . . . . .
 [7,] . . . . . . . . . .
 [8,] . . . . . . . . . .
 [9,] . . . . . . . . . .
[10,] . . . . . . . . . .
```

We next convert distance to similarity and retain weights only for observed network ties. Multiplication by the original adjacency matrix ensures that non-ties remain zero, while the $1/(1+d)$ transformation assigns equal-age ties a weight of one:

```r
# Transforming age distance into similarity on observed ties
exp_age_homo_graph <- kfam_diffnet$graph[[1]] *
  (1 / (1 + as.matrix(age_distance_graph)))

exp_age_homo <- exposure(
  kfam_diffnet,
  alt.graph = exp_age_homo_graph,
  valued = TRUE
)
```

```
Warning in exposure(kfam_diffnet, alt.graph = exp_age_homo_graph, valued =
TRUE): When -alt.graph- is static, will be repeated "t" times to fit the data.
```

The warning from `exposure()` explains that the static alternative graph will be repeated for all T time points. This is intentional: the Korean Family Planning example uses the same observed nomination network throughout the study. The next code chunk gives a glimpse of the exposure values we have calculated:

```
# Visualizing exposure at time 5
cbind(
  "Cohesion"       = exp_cohesive[, 5],
  "SE"             = exp_se[, 5],
  "Attr. weighted" = exp_age[, 5],
  "Homophilic"     = exp_age_homo[, 5]
) |> head(10)
```

```
     Cohesion      SE Attr. weighted Homophilic
201       1.0 0.03250         1.0000     1.0000
202       0.5 0.03207         0.4795     0.7143
203       0.0 0.03833         0.0000     0.0000
204       1.0 0.03178         1.0000     1.0000
205       0.0 0.04161         0.0000     0.0000
206       0.5 0.03224         0.5000     0.5435
207       0.0 0.27304         0.0000     0.0000
209       0.0 0.27304         0.0000     0.0000
210       1.0 0.03830         1.0000     1.0000
2101      0.0 0.28641         0.0000     0.0000
```

In sum, the `exposure()` function provides considerable flexibility for calculating a wide range of exposures. Computing exposures is a fundamental task in the study of diffusion networks. A more in-depth treatment is given in (Valente 2024).

Fitting a lagged exposure model

A natural step after computing exposures is estimating lagged exposure models. A common approach is a discrete-time logistic regression that predicts adoption as a function of node characteristics and lagged exposure. The `diffreg()` function facilitates this analysis. When `exposure` is included on its own in the model formula, `diffreg()` automatically uses a one-period lag. To illustrate an important modeling safeguard, the deliberately simplified baseline model below omits period indicators. It therefore displays a warning about `per`, which we explain immediately afterward. The following model uses the default one-period exposure lag:

```
model_baseline <- diffreg(kfam_diffnet ~ exposure)
```

```
Warning: The variable `per` is not included in the model. This can bias the
`exposure` as adoption naturally increases over time.
```

```
summary(model_baseline)
```

```
Call:
glm(formula = Adopt ~ exposure, family = binomial(link = "logit"),
    data = dat, subset = ifelse(is.na(toa), TRUE, toa >= per))

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   -2.655      0.194  -13.68   <2e-16 ***
exposure       0.606      0.376    1.61     0.11
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 391.93  on 709  degrees of freedom
Residual deviance: 389.43  on 708  degrees of freedom
  (113 observations deleted due to missingness)
AIC: 393.4

Number of Fisher Scoring iterations: 5
```

The `diffreg()` function uses R's `glm()` function after expanding the data into person-period observations and retaining each person only while they are at risk of adopting, analogous to a discrete-time event-history analysis. Adoption at time t is modeled using network exposure at time $t-1$. Because the predictor is lagged, one person's adoption at time t is not used contemporaneously to predict another person's adoption at the same time. Under the usual conditional-independence assumptions of the specified logistic regression, the model can therefore be estimated using standard GLM inference; the fact that exposure is computed from a network does not by itself invalidate the regression. As in other regression settings, researchers may add fixed effects, random effects, or other adjustments when their substantive design requires them.

The baseline exposure coefficient is 0.606 on the log-odds scale ($p = 0.107$). Thus, a 0.10 increase in the proportion of adopting contacts is associated with a multiplicative change of

$\exp(0.10 \times 0.606) = 1.06$ in the odds of adoption, or approximately 6% higher odds. The estimate is not statistically distinguishable from no association at conventional levels. This result is only illustrative because the model omits the baseline adoption hazard. The displayed warning explains that adoption can change over time, so omitting the period variable `per` can attribute a common time trend to exposure. The next specification includes period indicators and individual characteristics and explicitly shows the one-period exposure lag:

```
model_adjusted <- suppressWarnings(diffreg(
  kfam_diffnet ~
    # Adding age and a factor for village
    age + factor(village) + factor(per) +
    # Lagged exposure that uses the alternative graph
    exposure(
        lags = 1,
        alt.graph = exp_age_homo_graph,
        valued = TRUE
        )
  ))

summary(model_adjusted)
```

```
Call:
glm(formula = Adopt ~ exposure + age + factor(village) + factor(per),
    family = binomial(link = "logit"), data = dat, subset = ifelse(is.na(toa),
        TRUE, toa >= per))

Coefficients:
                  Estimate Std. Error z value Pr(>|z|)
(Intercept)        -3.6662     0.7834   -4.68  2.9e-06 ***
exposure            0.2845     0.4535    0.63   0.5305
age                 0.0557     0.0207    2.69   0.0071 **
factor(village)21  -0.7011     0.2933   -2.39   0.0168 *
factor(per)3       -0.4627     0.5139   -0.90   0.3678
factor(per)4       -0.3678     0.5286   -0.70   0.4866
factor(per)5       -0.6431     0.5965   -1.08   0.2810
factor(per)6       -1.0860     0.7015   -1.55   0.1216
factor(per)7       -0.7391     0.6438   -1.15   0.2510
factor(per)8       -0.4481     0.6124   -0.73   0.4643
factor(per)9        0.0237     0.5706    0.04   0.9668
factor(per)10      -0.0361     0.5982   -0.06   0.9519
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 391.93  on 709  degrees of freedom
Residual deviance: 368.62  on 698  degrees of freedom
  (113 observations deleted due to missingness)
AIC: 392.6

Number of Fisher Scoring iterations: 6
```

Arguments can be passed to `exposure()` directly through the model formula. In the adjusted model, the age-homophilic exposure coefficient is 0.284 ($p = 0.53$). A 0.10 increase in this exposure corresponds to an estimated odds ratio of $\exp(0.10 \times 0.284) = 1.03$, but the confidence interval includes no association. We therefore do not find a statistically detectable association between lagged age-homophilic exposure and adoption under this specification.

Age is positively associated with adoption: each additional year of age corresponds to an estimated odds ratio of $\exp(0.056) = 1.06$, or about 6% higher odds. The coefficient for village 21 is -0.701, corresponding to an odds ratio of $\exp(-0.701) = 0.50$ relative to village 2. Thus, conditional on the other model terms, respondents in village 21 have approximately half the odds of adoption of respondents in village 2.

Individual period coefficients compare each period with the reference period and need not be statistically significant one at a time for time to matter jointly. We can test the period indicators as a group using a likelihood-ratio test:

```r
drop1(model_adjusted, test = "Chisq")["factor(per)", ]
```

```
Single term deletions

Model:
Adopt ~ exposure + age + factor(village) + factor(per)
            Df Deviance AIC  LRT Pr(>Chi)
factor(per)  8      374 382 5.33     0.72
```

After removing the artificial terminal period, the joint likelihood-ratio test does not reject a constant baseline adoption hazard over the observed periods ($p = 0.72$). This differs from the analysis that incorrectly treated the forced terminal adoptions as observed events: those entries generated a very large period-11 coefficient and made the period indicators appear jointly significant. The revised result illustrates why censoring the terminal cases and removing period 11 are important.

More details about what happens when calling the `diffreg()` function can be found in the library's manual.

Simulating diffusion networks

Diffusion processes in networks are simulated using `rdiffnet()`, which supports options such as graph topology, threshold levels, the proportion of initial adopters, the centrality of those seeds, and the number of time steps. The simulation algorithm is as follows:

1. If there is no graph data, a baseline graph is created,
2. the initial adopters are identified,
3. a threshold value is assigned to each node,
4. if requested, a sequence of t networks is created, and
5. new adopters are selected at times $t > 1$ based on each node's exposure at time $t - 1$. If that exposure exceeds the node's threshold, the node adopts; otherwise, it remains a non-adopter.

The function requires only two inputs: the number of nodes (`n`) and the number of time steps (`t`). By default, it generates a scale-free graph with random rewiring at each time step, sets the proportion of initial adopters to 0.05, and draws a threshold for each node from a uniform distribution. This example shows the default settings for a diffusion process with 200 nodes and 10 time steps:

```
# Make the simulated network reproducible.
set.seed(331)

# Give the dimensions names so they can be reused below.
n_nodes <- 200
n_periods <- 10

diffnet_1 <- rdiffnet(
  # Required inputs
  n = n_nodes,
  t = n_periods,
  # Optional arguments
  seed.nodes     = "random",
  seed.p.adopt   = 0.05,
  seed.graph     = "scale-free",
  rgraph.args    = list(),
  rewire         = TRUE,
```

```
  rewire.args    = list(),
  threshold.dist = runif(n_nodes),
  exposure.args  = list(),
  name           = "A diffusion network",
  behavior       = "Random contagion",
  stop.no.diff   = TRUE,
  disadopt       = NULL
)
```

This is equivalent to calling `rdiffnet(200, 10)`:

```
set.seed(331)
rdiffnet(200, 10)
```

Since the simulation process is inherently random, we use the function `set.seed()` to ensure reproducibility.

> **Tip**
>
> Although the threshold model is deterministic conditional on its inputs, `rdiffnet()` calls `runif()` to generate threshold values by default, making the overall simulation stochastic. With fixed thresholds, a fixed network, and fixed seed nodes, it would be deterministic.

`rdiffnet()` supports thresholds specified as a fixed value for all nodes, a vector of node-specific values, or a function. The proportion of initial adopters (`seed.p.adopt`) is used together with `seed.nodes`. Supported character values for `seed.nodes` are `"random"`, `"central"`, and `"marginal"`. If a vector of node indices is supplied instead, `rdiffnet()` ignores `seed.p.adopt` and uses those nodes as the initial adopters.

The `rdiffnet()` function also supports the use of a pre-defined graph as input. The following code shows how to simulate a diffusion process using a Watts-Strogatz graph:

```
# Creating a graph based on the Watts-Strogatz model
set.seed(121)
graph <- rgraph_ws(200, 10, p = .3)

# Run the simulation
diffnet_2 <- rdiffnet(
  t              = 10,
  seed.graph     = graph,
  seed.p.adopt   = 0.1,
  threshold.dist = runif(200, .3, .5)
)
```

Since we are passing an actual network to the `seed.graph` argument, the `n` argument is no longer needed.

Multiadoption

Starting with version 1.23.0, **netdiffuseR** supports simulating multi- and dis-adoption diffusion processes. To study a multi-adoption process, you can pass a `list` as the `seed.p.adopt` parameter. Here is a simple example:

```
set.seed(1231)

diffnet_3 <- rdiffnet(
  200, 10,
  seed.p.adopt = list(0.1, 0.15),
  seed.nodes   = list("random", "random"),
  behavior     = list("Behavior 1", "Behavior 2")
)
```

```
Message: Multi-diffusion behavior simulation selected. Number of behaviors:  2
```

```
diffnet_3
```

```
Dynamic network of class -diffnet-
 Name               : A diffusion network
 Behavior           : Behavior 1, Behavior 2
 # of nodes         : 200 (1, 2, 3, 4, 5, 6, 7, 8, ...)
 # of time periods  : 10 (1 - 10)
 Type               : directed
 Num of behaviors   : 2
 Prevalence         : 0.29, 0.97
 Static attributes  : real_threshold.1, real_threshold.2 (2)
 Dynamic attributes : -
```

The output identifies the calculation as a multi-diffusion simulation and reports two behaviors, along with a final prevalence for each. `rdiffnet()` simulates as many behaviors as there are entries in the `seed.p.adopt` list.

> **Tip**
>
> In the current implementation of `rdiffnet()`, the multi-adoption module simulates behaviors independently. That is, the code above would be equivalent to simulating the same behavior twice. More complicated models in which behaviors are interdependent are supported via the dis-adoption parameter.

`rdiffnet()`'s defaults will replicate the simulation parameters across behaviors. Nonetheless, we can use lists to specify different parameters for each behavior. For example, the following code simulates two behaviors with different initial adopters, threshold distributions, seed nodes, and labels for the behaviors:

```
set.seed(1232)
diffnet_4 <- rdiffnet(
  200, 10,
  seed.p.adopt = list(0.1, 0.15),
  threshold.dist = list(
    runif(200, .3, .5),
    runif(200, .2, .4)
  ),
  seed.nodes = list("central", "random"),
  behavior   = list("tobacco", "alcohol")
)
```

```
Message: Multi-diffusion behavior simulation selected. Number of behaviors:  2
```

```
diffnet_4
```

```
Dynamic network of class -diffnet-
 Name               : A diffusion network
 Behavior           : tobacco, alcohol
 # of nodes         : 200 (1, 2, 3, 4, 5, 6, 7, 8, ...)
 # of time periods  : 10 (1 - 10)
 Type               : directed
 Num of behaviors   : 2
 Prevalence         : 0.99, 0.77
 Static attributes  : real_threshold.1, real_threshold.2 (2)
 Dynamic attributes : -
```

In this particular example, we ran `rdiffnet()` with most of the parameters being in a `list`. The reader is invited to look at other types of possible inputs in the `rdiffnet()` documentation.

As in the previous section, we can provide a specific network as input. Besides fixed networks such as the small-world example, `rdiffnet()` supports `diffnet` objects. It then uses the object's graph as the seed graph and its observed periods as the simulation time points. Here is an example using the Korean Family Planning network:

```
set.seed(1233)
diffnet_5 <- rdiffnet(
  seed.graph     = kfam_diffnet,
  seed.p.adopt   = list(0.1, 0.15),
  seed.nodes     = list("random", "random"),
  threshold.dist = runif(nvertices(kfam_diffnet), .3, .5),
  behavior       = list("Behavior 1", "Behavior 2")
)
```

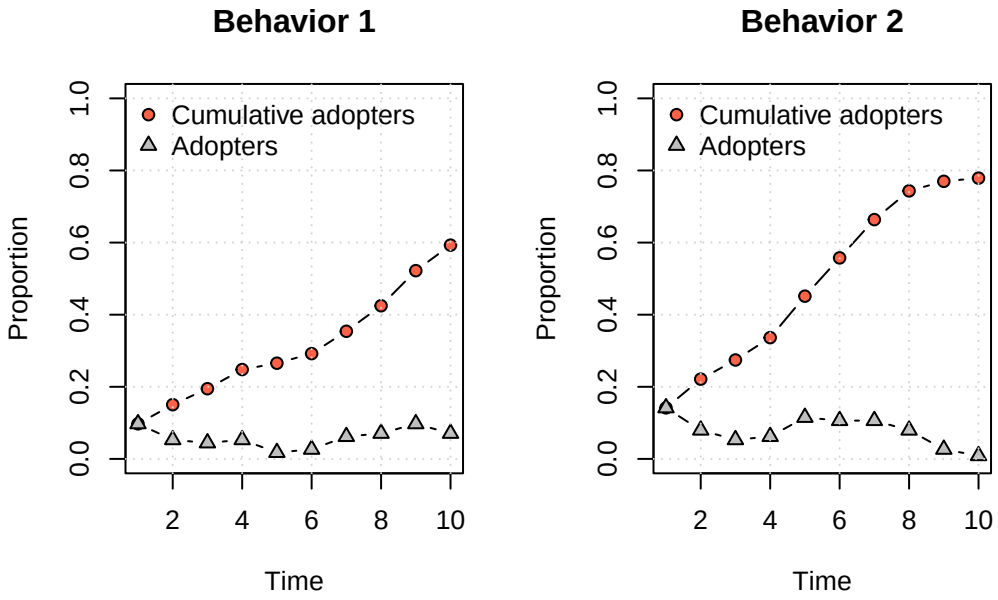
```
Message: Multi-diffusion behavior simulation selected. Number of behaviors:  2
```

```
diffnet_5
```

```
Dynamic network of class -diffnet-
 Name               : A diffusion network
 Behavior           : Behavior 1, Behavior 2
 # of nodes         : 113 (1, 2, 3, 4, 5, 6, 7, 8, ...)
 # of time periods  : 10 (1 - 10)
 Type               : directed
 Num of behaviors   : 2
 Prevalence         : 0.59, 0.78
 Static attributes  : real_threshold.1, real_threshold.2 (2)
 Dynamic attributes : -
```

To visualize a diffusion process with more than one behavior, `split_behaviors()` separates the object into one `diffnet` object per behavior. We then use `plot_adopters()` for each behavior and `par()` to arrange both plots in a single window:

```
diffnets_5 <- split_behaviors(diffnet_5)
op <- par(mfrow = c(1, 2), cex = .8)
plot_adopters(diffnets_5[[1]], main = "Behavior 1")
plot_adopters(diffnets_5[[2]], main = "Behavior 2")
```




```
par(op)
```

Notice the `op` object that stores the original graphical parameters. We use it to reset the graphical parameters after plotting the two behaviors.

Disadoption

The disadoption feature introduced in netdiffuseR version 1.23.0 enables the study of diffusion processes in which adoption can be reversed. Disadoption is important for studying competition between products or beliefs (Lehmann and Parker 2017). The `disadopt` argument of `rdiffnet()` accepts a user-defined disadoption function, supporting tests of theoretical models for disadoption (Alipour et al. 2024). Such a function must satisfy three requirements:

1. It must be a function that receives three arguments: `expo`, `cumadopt`, and `time`.
2. It must return one vector per behavior, identifying the nodes that drop each behavior. For example, if only node 5 drops the first of three behaviors, it should return `list(5L, integer(), integer())`.
3. If no nodes disadopt any of the n behaviors, the function must return a list of n empty integer vectors: `replicate(n, integer(), simplify = FALSE)`.

A template for a disadoption function, which currently returns no disadoption, follows:

```
disadoption_function <- function(expo, cumadopt, time) {
  replicate(dim(cumadopt)[3], integer(), simplify = FALSE)
}
```

The following code shows how to build a disadoption function that randomly selects 10% of the adopters at time `t - 1`:

```
random_dis <- function(expo, cumadopt, time) {

  # Number of behaviors
  num_of_behaviors <- dim(cumadopt)[3]

  # Making room for the disadopted nodes
  list_disadopt <- replicate(
    num_of_behaviors,
    integer(),
    simplify = FALSE
  )

  # Iterating through the behaviors
  for (q in 1:num_of_behaviors) {

    # Identifying the adopters at time t-1
    adopters_old <- which(cumadopt[, time - 1, q] == 1)

    if (length(adopters_old) != 0) {

      # Selecting 10% of adopters to disadopt
      list_disadopt[[q]] <- sample(
        adopters_old,
        ceiling(0.10 * length(adopters_old)
        )
      )

    }

  }

  list_disadopt

}
```

It is worth highlighting a few things from the code:

1. The `expo` argument is the entire exposure *array*. This means that it has three dimensions: the first dimension is the number of nodes, the second is the number of time points, and the third is the number of behaviors.
2. The `cumadopt` argument is the cumulative adoption array. It has the same dimensions as `expo`. The value of `cumadopt[i, t, q]` is 1 if node `i` has adopted behavior `q` at time `t`.
3. The `time` argument is the current time point in the simulation. This allows the function to know when the disadopt function is being called.
4. The code `which(cumadopt[, time - 1, q] == 1)` identifies the nodes whose `cumadopt` entry equals 1 at time $t-1$.

To simulate a diffusion process with disadoption, we can use the `rdiffnet()` function as follows:

```
set.seed(1234)
diffnet_6  <- rdiffnet(
  seed.graph   = graph,
  t            = 10,
  disadopt     = random_dis,
  seed.p.adopt = 0.1
  )

diffnet_6
```

```
Dynamic network of class -diffnet-
 Name               : A diffusion network
 Behavior           : Random contagion
 # of nodes         : 200 (1, 2, 3, 4, 5, 6, 7, 8, ...)
 # of time periods  : 10 (1 - 10)
 Type               : directed
 Num of behaviors   : 1
 Final prevalence   : 0.43
 Static attributes  : real_threshold (1)
 Dynamic attributes : -
```

Using a `disadopt` function, we can build more complex models with competing behaviors. For instance, the following rule prevents nodes from retaining both behaviors simultaneously by making double adopters drop behavior 1:

$$\text{Disadopt } 1_{it} = \begin{cases} Yes, & \text{If behavior 2 has adopted} \\ \text{No}, & \text{otherwise.} \end{cases}$$

The following code shows how to build such a function:

```
one_only <- function(expo, cumadopt, time) {

  # Identifying double adopters
  ids <- which(apply(cumadopt[, time, ], 1, sum) == 2)

  if (length(ids) == 0)
    return(list(integer(), integer()))

  # Otherwise, make them pick one (in this case, we prefer the second)
  list(ids, integer())

}
```

Of the code above, we can highlight the following:

1. We identify the number of behaviors adopted by each individual at time `t` using `apply(cumadopt[, time, ], 1, sum)`. In a two-behavior model, this returns 0 for no adoption, 1 for one behavior, or 2 for both behaviors.
2. The `which()` function is used to identify the nodes adopting both behaviors (after calling `apply()`).
3. If there are no double adopters, the function returns a list with empty vectors.

Let's simulate a diffusion process with the disadoption function `one_only`:

```
set.seed(331)
diffnet_7 <- rdiffnet(
  200, 10,
  disadopt     = one_only,
  seed.p.adopt = list(0.1, 0.1),
  seed.nodes   = list("random", "random"),
  behavior     = list("Behavior 1", "Behavior 2")
)
```

```
Message: Multi-diffusion behavior simulation selected. Number of behaviors:  2
```

```
diffnet_7
```

```
Dynamic network of class -diffnet-
 Name               : A diffusion network
 Behavior           : Behavior 1, Behavior 2
 # of nodes         : 200 (1, 2, 3, 4, 5, 6, 7, 8, ...)
 # of time periods  : 10 (1 - 10)
 Type               : directed
 Num of behaviors   : 2
 Prevalence         : 0.28, 0.42
 Static attributes  : real_threshold.1, real_threshold.2 (2)
 Dynamic attributes : -
```

Finally, we can demonstrate that each node adopted at most one behavior by taking the cumulative adoption matrix at the last time point and checking whether any node adopted both. The `toa_mat()` function extracts the adoption matrices from the model:

```
toas <- toa_mat(diffnet_7)

# Putting the two behaviors together
adopted <-  cbind(
  toas[[1]]$cumadopt[, 10],
  toas[[2]]$cumadopt[, 10]
)

# Looking at the first 5
head(adopted, 5)
```

```
  [,1] [,2]
1    0    0
2    0    1
3    1    0
4    1    0
5    0    1
```

```
# Tabulating side by side
table(adopted[, 1], adopted[, 2])
```

```
   0  1
```

```
  0 61 84
  1 55  0
```

As expected, there's no entry in the table in which both behaviors were adopted by the same node. Using the `disadopt` function, we can build (and study) increasingly complex models of network diffusion.

Conclusion

This paper has introduced the netdiffuseR R package, which facilitates the estimation and simulation of contagion and diffusion processes on networks. The tutorial provides reproducible examples that readers can adapt to the classical datasets included with the package or to their own data. Its empirical sections demonstrate the calculation and interpretation of network exposure, including extensions based on structural equivalence, attributes, and homophily. Its simulation sections provide tools for theoretical testing of diffusion processes.

We also provide examples of multi-diffusion processes in which more than one idea or behavior spreads through a network. The Korean Family Planning study, for example, records multiple family-planning choices over time. The disadoption examples show how to represent processes in which an individual adopts a behavior, subsequently discontinues it, and may later resume it. Although few available datasets capture these dynamics in detail, the package supplies tools for studying them as such data become available.

We hope that by providing these illustrative examples, others will be able to use netdiffuseR in their work. Diffusion processes are ubiquitous in science and everyday life. The diffusion paradigm was first established by Ryan and Gross in 1943 (Valente and Rogers 1995) but it continues to grow and evolve, and our hope is that this publication will further encourage both growth and evolution.

Acknowledgements

This work was supported by the National Institute of Drug Abuse, the National Institutes of Health (NIH), grant number R01-DA051843. We would also like to thank an anonymous reviewer for their comments and suggestions that improved this paper.

References

Alipour, Fahimeh, Fedor Dokshin, Zeinab Maleki, Yunya Song, and Pouria Ramazi. 2024. "Enough but Not Too Many: A Bi-Threshold Model for Behavioral Diffusion." *PNAS Nexus* 3 (10): pgae428. https://doi.org/10.1093/pnasnexus/pgae428.

Allaire, J. J., Charles Teague, Carlos Scheidegger, Yihui Xie, Christophe Dervieux, and Gordon Woodhull. 2025. *Quarto*. V. 1.7. Released April. https://doi.org/10.5281/zenodo.5960048.

Aral, Sinan, and Christos Nicolaides. 2017. "Exercise Contagion in a Global Social Network." *Nature Communications* 8 (1): 14753. https://doi.org/10.1038/ncomms14753.

Bass, Frank M. 1969. "A New Product Growth for Model Consumer Durables." *Management Science* 15 (5): 215–27. https://doi.org/10.1287/mnsc.15.5.215.

Berry, George, Christopher J. Cameron, Patrick Park, and Michael Macy. 2019. "The Opacity Problem in Social Contagion." *Social Networks* 56 (January): 93–101. https://doi.org/10.1016/j.socnet.2018.09.001.

Bongaarts, John, and Susan Cotts Watkins. 1996. "Social Interactions and Contemporary Fertility Transitions." *Population and Development Review* 22 (4): 639. https://doi.org/10.2307/2137804.

Burt, Ronald S. 1987. "Social Contagion and Innovation: Cohesion Versus Structural Equivalence." *American Journal of Sociology* 92 (6): 1287–335. https://doi.org/10.1086/228667.

Christakis, Nicholas A., and James H. Fowler. 2007. "The Spread of Obesity in a Large Social Network over 32 Years." *New England Journal of Medicine* 357 (4): 370–79. https://doi.org/10.1056/NEJMsa066082.

Coleman, James Samuel, Elihu Katz, and Herbert Menzel. 1966. *Medical Innovation; a Diffusion Study / [by] James s. Coleman, Elihu Katz [and] Herbert Menzel. Foreword by Joseph a. Precker.* Bobbs-Merrill Co.

Fujimoto, Kayo, and Thomas W. Valente. 2013. "Alcohol Peer Influence of Participating in Organized School Activities: A Network Approach." *Health Psychology* 32 (10): 1084–92. https://doi.org/10.1037/a0029466.

Hoffman, Beth R., Steve Sussman, Jennifer B. Unger, and Thomas W. Valente. 2006. "Peer Influences on Adolescent Cigarette Smoking: A Theoretical Review of the Literature." *Substance Use & Misuse* 41 (1): 103–55. https://doi.org/10.1080/10826080500368892.

Iyengar, Raghuram, Christophe Van Den Bulte, and Thomas W. Valente. 2011. "Opinion Leadership and Social Contagion in New Product Diffusion." *Marketing Science* 30 (2): 195–212. https://doi.org/10.1287/mksc.1100.0566.

Katz, Elihu. 1957. "The Two-Step Flow of Communication: An Up-To-Date Report on an Hypothesis." *Public Opinion Quarterly* 21 (1, Anniversary Issue Devoted to Twenty Years of Public Opinion Research): 61. https://doi.org/10.1086/266687.

Lehmann, Donald R., and Jeffrey R. Parker. 2017. "Disadoption." *AMS Review* 7 (1-2): 36–51. https://doi.org/10.1007/s13162-017-0093-8.

Light, John M., Charlotte C. Greenan, Julie C. Rusby, Kimberley M. Nies, and Tom A. B. Snijders. 2013. "Onset to First Alcohol Use in Early Adolescence: A Network Diffusion Model." *Journal of Research on Adolescence* 23 (3): 487–99. https://doi.org/10.1111/jora.12064.

Moran, P. A. P. 1950. "Notes on Continuous Stochastic Phenomena." *Biometrika* 37 (1/2): 17. https://doi.org/10.2307/2332142.

R Core Team. 2025. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. https://www.R-project.org/.

Rogers, Everett M. 2003. *Diffusion of Innovations*. Fifth edition. Free Press.

Rogers, Everett M., and D. Lawrence Kincaid. 1981. *Communication Networks: Toward a New Paradigm for Research*. Free Press ; Collier Macmillan.

Valente, Thomas W. 1993. "Diffusion of Innovations and Policy Decision-Making." *Journal of Communication* 43 (1): 30–45. https://doi.org/10.1111/j.1460-2466.1993.tb01247.x.

Valente, Thomas W. 1995. *Network Models of the Diffusion of Innovations*. 2nd ed. Hampton Press.

Valente, Thomas W. 1996. "Social Network Thresholds in the Diffusion of Innovations." *Social Networks* 18 (1): 69–89. https://doi.org/10.1016/0378-8733(95)00256-1.

Valente, Thomas W. 2010. *Social Networks and Health: Models, Methods, and Applications*. 1st ed. Oxford University PressNew York. https://doi.org/10.1093/acprof:oso/9780195301014.001.0001.

Valente, Thomas W. 2024. "Contagion and Interpersonal Influence: Distinguishing Mechanisms of Behavior Change Using Social Network Theory." *Connections* 0 (0): –. https://doi.org/doi:10.21307/connections-2019.041.

Valente, Thomas W., Stephanie R. Dyal, Kar-Hai Chu, Heather Wipfli, and Kayo Fujimoto. 2015. "Diffusion of Innovations Theory Applied to Global Tobacco Control Treaty Ratification." *Social Science & Medicine* 145 (November): 89–97. https://doi.org/10.1016/j.socscimed.2015.10.001.

Valente, Thomas W., and Everett M. Rogers. 1995. "The Origins and Development of the Diffusion of Innovations Paradigm as an Example of Scientific Growth." *Science*

Communication 16 (3): 242–73. https://doi.org/10.1177/1075547095016003002.

Valente, Thomas W., and Walter P. Saba. 1998. "Mass Media and Interpersonal Influence in a Reproductive Health Communication Campaign in Bolivia." *Communication Research* 25 (1): 96–124. https://doi.org/10.1177/009365098025001004.

Valente, Thomas W., and George G. Vega Yon. 2020. "Diffusion/Contagion Processes on Social Networks." *Health Education & Behavior* 47 (2): 235–48. https://doi.org/10.1177/1090198120901497.

Valente, Thomas W., Susan C. Watkins, Miriam N. Jato, Ariane Van Der Straten, and Louis-Philippe M. Tsitsol. 1997. "Social Network Associations with Contraceptive Use Among Cameroonian Women in Voluntary Associations." *Social Science & Medicine* 45 (5): 677–87. https://doi.org/10.1016/S0277-9536(96)00385-1.

Vega Yon, George, Anibal Olivera Morales, and Thomas Valente. 2026. *netdiffuseR: Analysis of Diffusion and Contagion Processes on Networks*. https://doi.org/10.5281/zenodo.1039317.

Yamagata, Yoshiki, Jue Yang, and Joseph Galaskiewicz. 2017. "State power and diffusion processes in the ratification of global environmental treaties, 1981–2008." *International Environmental Agreements: Politics, Law and Economics* 17 (4): 501–29. https://doi.org/10.1007/s10784-016-9332-y.